



Métricas de Desempenho em Sistemas Paralelos

Aula 04

Prof. Paulo Bressan



Sumário

1. Introdução ao Desempenho em Sistemas Paralelos

Conceito de desempenho, importância da medição em ambientes paralelos e tipos de sobrecarga.

2. Métricas Clássicas de Desempenho

Tempo de resposta, throughput e tempo de execução — definições, diferenças e comparação sequencial x paralelo.

3. Leis Fundamentais e Overheads

Speedup, eficiência, fontes de overhead, Lei de Amdahl, escalabilidade forte e fraca, Lei de Gustafson-Barsis e métrica de Karp-Flatt.

4. Estudos de Caso e Práticas de Avaliação

Análise de escalabilidade, redução paralela, exercícios e métricas de redundância, qualidade e utilização.

Conceito de desempenho em sistemas computacionais

Definição de desempenho

Desempenho é medido pela capacidade de um sistema resolver um problema mais rapidamente (ou processar mais trabalho) à medida que se aumentam os recursos computacionais — processadores, núcleos ou nós — trabalhando simultaneamente sobre partes do mesmo problema.

Métricas de desempenho

Tempo de execução, *throughput* (vazão ou taxa de transferência) e latência são medidas comuns para avaliar o desempenho de sistemas computacionais.

Importância em sistemas paralelos

Aproveitar múltiplos processadores é fundamental para melhorar o desempenho em sistemas paralelos.



Importância da medição e análise em ambientes paralelos

Identificação de Gargalos

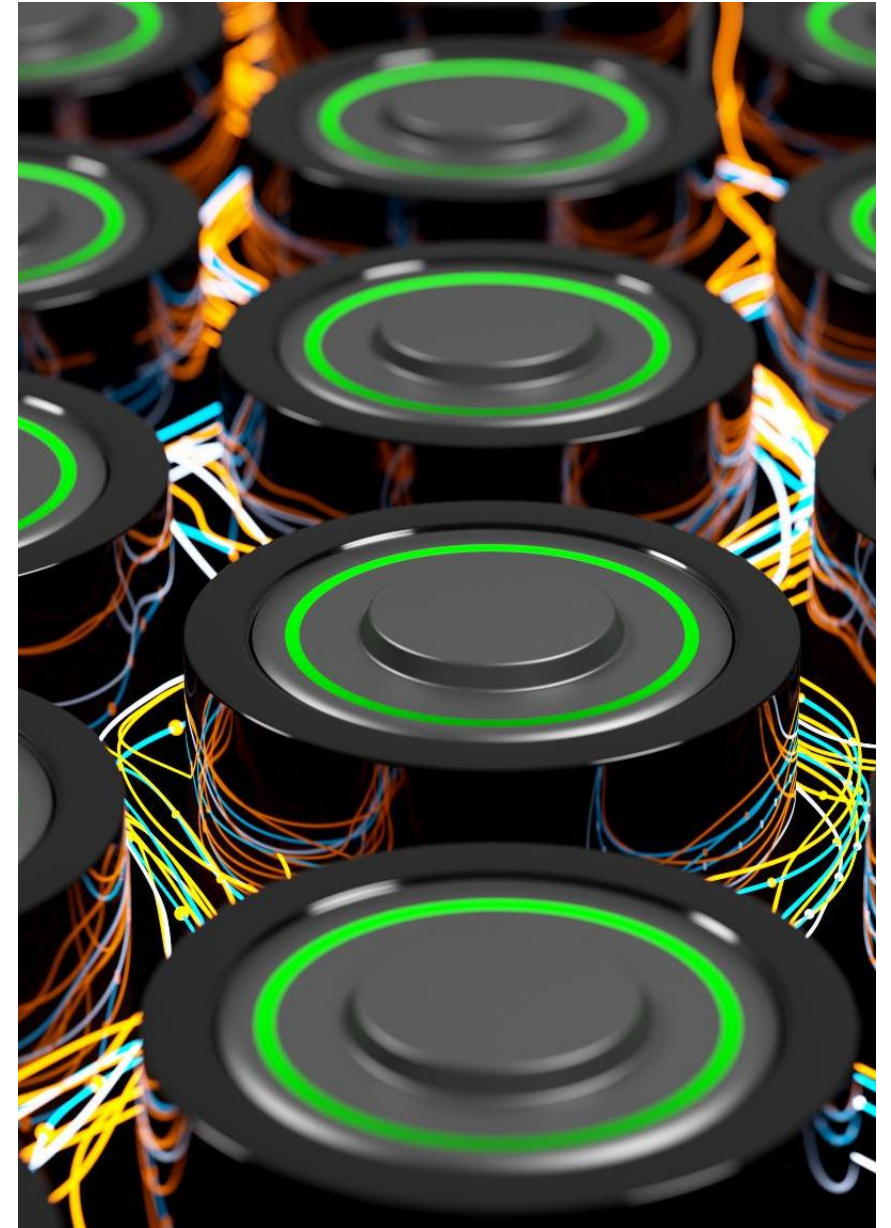
A medição permite detectar pontos críticos que limitam o desempenho em sistemas paralelos.

Otimização de Recursos

Analisar dados ajuda a alocar hardware e energia de forma eficiente para evitar desperdícios.

Valor da Paralelização

Garantir que a paralelização melhore o sistema sem comprometer energia ou hardware é essencial.



Tipos de Sobrecarga em Sistemas Paralelos

Fatores que reduzem o ganho de desempenho da paralelização



Comunicação

Troca de dados entre processos em diferentes processadores ou nós da rede.



Sincronização

Espera em barreiras, locks e seções críticas para manter a consistência.



Contenção

Disputa por recursos compartilhados: memória, cache e barramento.



Balanceamento de Carga

Custo de distribuir tarefas evitando processadores ociosos ou sobrecarregados.



Gerenciamento de Threads

Criar, escalonar e destruir processos e threads de execução.



Replicação de Dados

Duplicar dados entre nós, consumindo memória extra e tempo de cópia.



Sobrecarga Algorítmica

Trabalho redundante para evitar dependências entre tarefas paralelas.

Tempo de resposta (Response Time)



Definição: É o tempo total gasto para atender a uma requisição ou executar uma tarefa individual, desde o início até o fim.

Foco: Experiência do **usuário** ou de uma única aplicação.

Unidade: Geralmente em **segundos**, **milissegundos** ou **ciclos de CPU**.

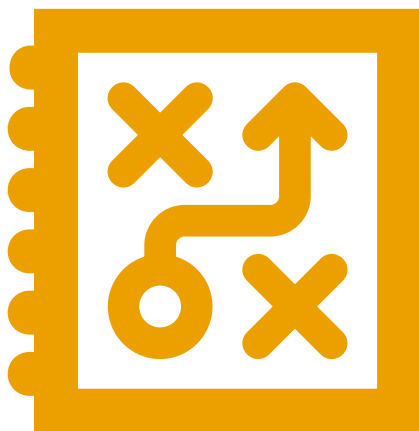
Exemplo:

- Em um sistema de banco de dados, o tempo entre o envio de uma consulta SQL e a devolução do resultado.
- Em um programa paralelo, quanto tempo leva para um processo terminar uma simulação.

Características:

- Importante para aplicações interativas ou tempo real (ex.: videogames, sistemas médicos, redes de telecomunicação).
- Diminuir o tempo de resposta nem sempre significa maior throughput.

Throughput (Vazão)



Definição: É a quantidade de tarefas ou requisições concluídas por unidade de tempo.

Foco: O sistema como um todo.

Unidade: Geralmente em **tarefas/segundo**, **transações/segundo**, **jobs/hora**.

Exemplo:

- Um servidor web que atende 10.000 requisições por segundo.
- Um cluster de HPC que finaliza 50 simulações de clima por hora.

Características:

- Importante em **ambientes de alto volume de processamento**, como data centers, sistemas de filas, redes de computadores.
- Pode aumentar mesmo que o tempo de resposta médio de cada tarefa seja maior (ex.: sistema em batch).

Tempo de Resposta e Throughput (Vazão) em Computação Paralela

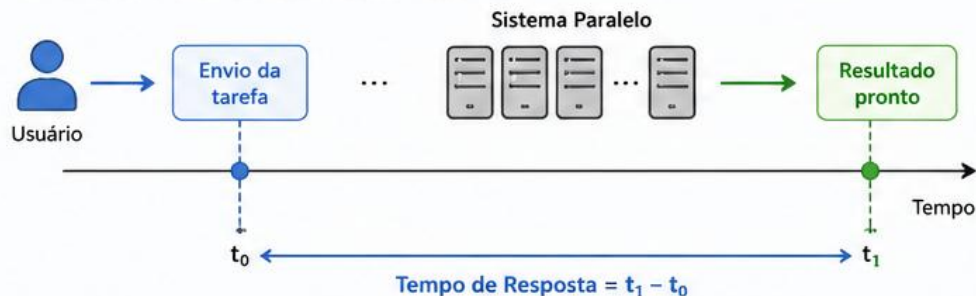
Duas métricas complementares para avaliar o desempenho de sistemas paralelos.

TEMPO DE RESPOSTA

É o **tempo decorrido** entre o envio de uma requisição (ou tarefa) até o recebimento da resposta (ou conclusão).

Exemplo

Um usuário envia uma tarefa e recebe o resultado.



Características

- Mede a experiência do usuário.
- Importante para aplicações interativas (ex.: sistemas online, consultas).
- Menor tempo de resposta é melhor.

Fatores que influenciam

- Tempo de processamento
- Comunicação e sincronização
- Fila de espera (contenção)
- Balanceamento de carga

Exemplo Numérico

Uma tarefa é enviada em $t_0 = 2,35$ s e o resultado chega em $t_1 = 2,85$ s.

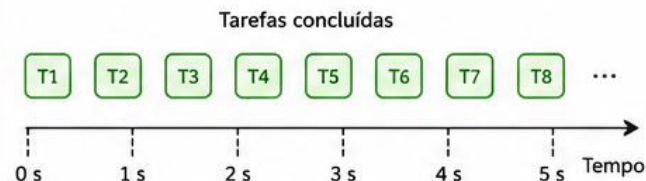
$$\text{Tempo de Resposta} = 2,85 - 2,35 = 0,50 \text{ segundos}$$

THROUGHPUT (VAZÃO)

É a **quantidade de trabalho** (tarefas) concluída por unidade de tempo.

Exemplo

O sistema processa várias tarefas ao longo do tempo.



Fórmula

$$\text{Throughput} = \frac{\text{Número de tarefas concluídas}}{\text{Tempo total}}$$

(tarefas/segundo)

Características

- Mede a **produtividade** do sistema.
- Importante para aplicações batch ou de alto processamento.
- Maior throughput é melhor.

Fatores que influenciam

- Grau de paralelismo
- Balanceamento de carga
- Comunicação e overhead
- Recursos disponíveis (CPU, memória, I/O)

Exemplo Numérico

Um sistema conclui 80 tarefas em 20 segundos.

$$\text{Throughput} = 80 / 20 = 4 \text{ tarefas/segundo}$$

Comparação e Quando Usar



Use TEMPO DE RESPOSTA quando:

- O foco é a experiência do usuário.
- As requisições são individuais ou de baixa frequência.
- Exemplos: sistemas web, bancos de dados, consultas interativas.



Use THROUGHPUT quando:

- O foco é processar o maior número possível de tarefas.
- As requisições são muitas e podem esperar na fila.
- Exemplos: processamento batch, simulações, análise de grandes volumes.

Resumo



Tempo de Resposta = rapidez



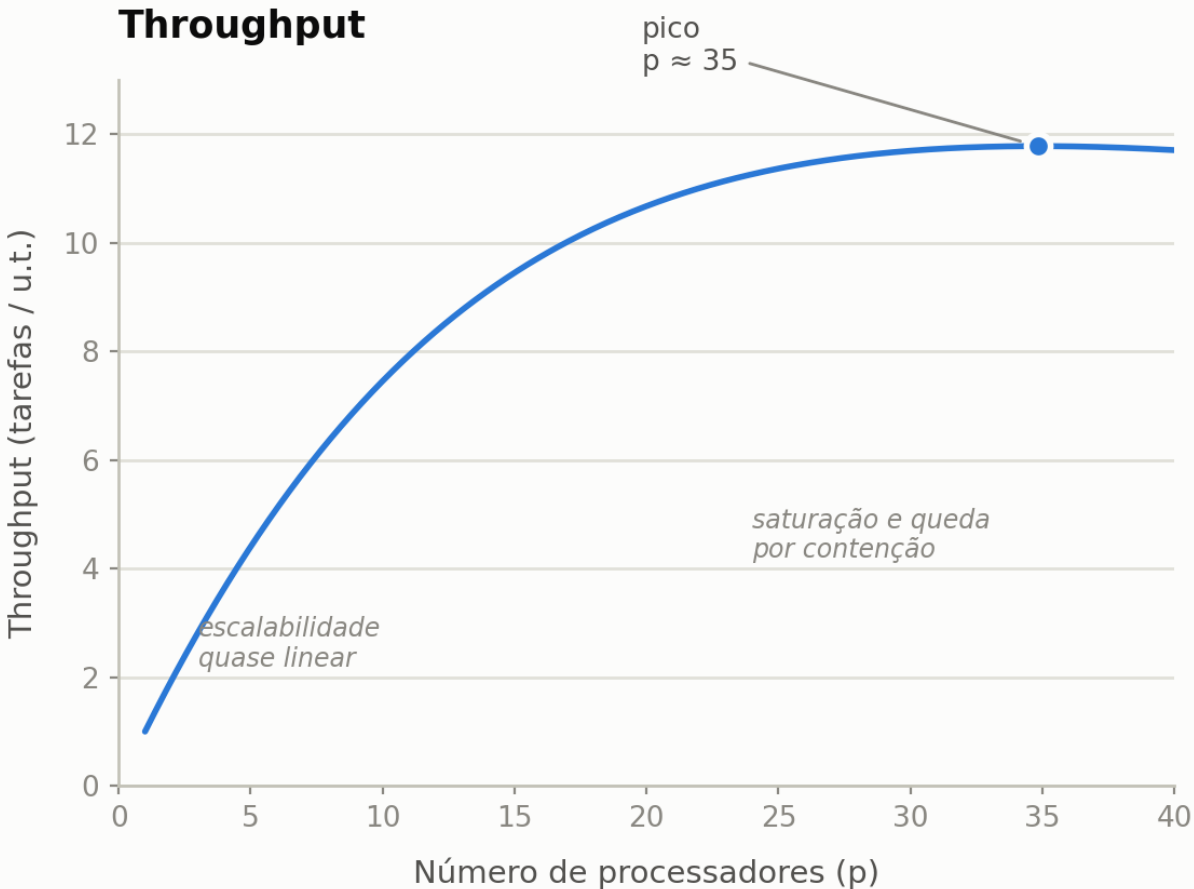
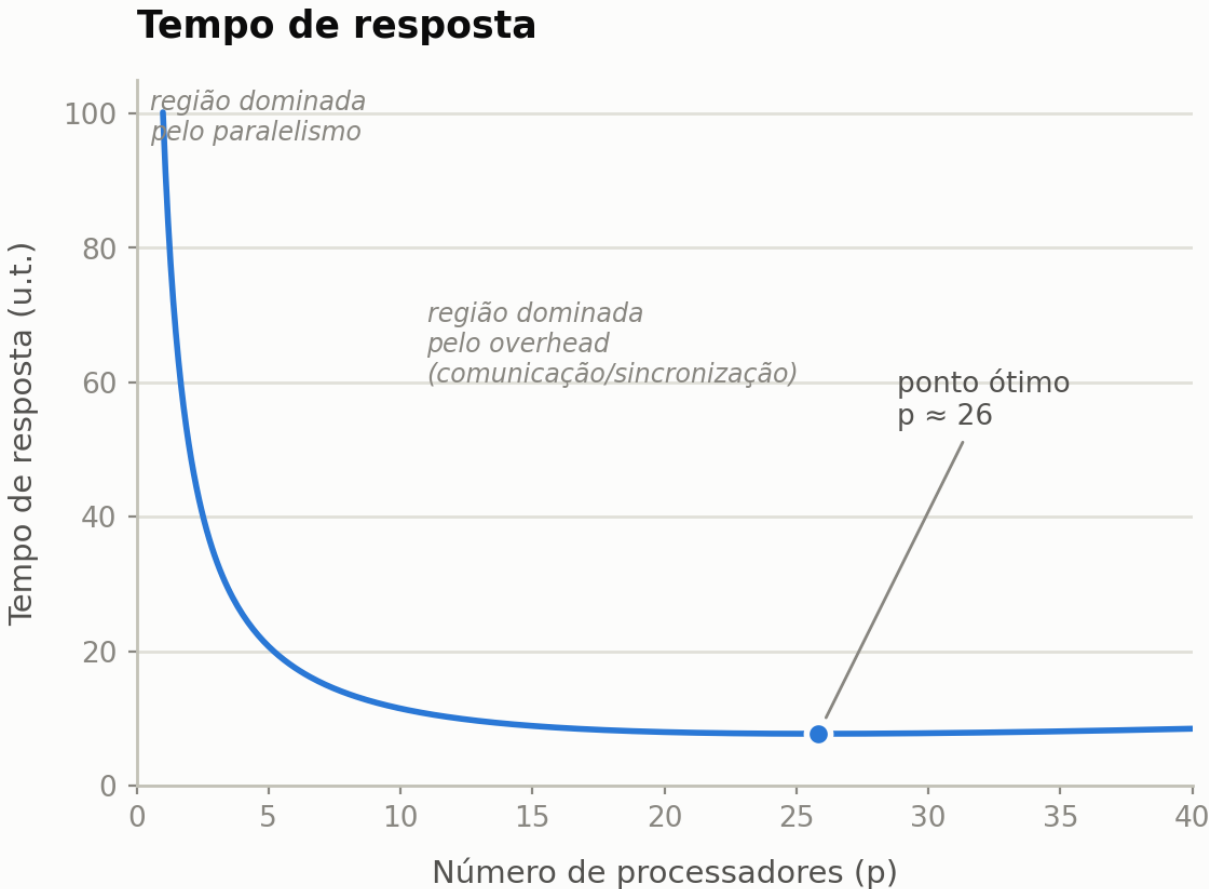
Throughput (Vazão) = produtividade



Em sistemas paralelos, o ideal é encontrar o equilíbrio: bom tempo de resposta para o usuário e alto throughput para o sistema.

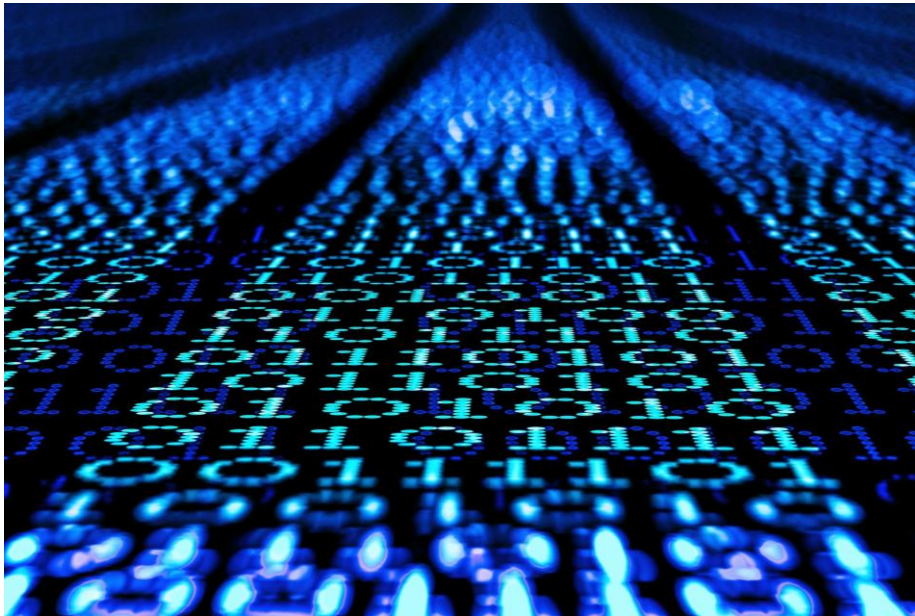
Tempo de resposta e throughput em função do número de processadores

Curvas ilustrativas (modelo conceitual) — comportamento típico em sistemas paralelos com overhead de comunicação, sincronização e contenção



Aspecto	Tempo de Resposta	Throughput
O que mede	Tempo para uma tarefa terminar	Quantidade de tarefas por tempo
Foco	Usuário / Tarefa individual	Sistema / Produção global
Unidade típica	Segundos, ms, ciclos	Tarefas/s, transações/s, jobs/hora
Aplicação típica	Sistemas interativos e tempo real	Servidores, clusters, data centers
Exemplo	Tempo para carregar uma página	Quantas páginas por segundo são servidas

Tempo de Execução: Sequencial x Paralelo



Definição

- **Tempo sequencial (T_{seq}):** tempo necessário para executar um programa em um único processador.
- **Tempo paralelo (T_{par}):** tempo para executar o mesmo programa em um sistema com múltiplos processadores.

Fatores que afetam T_{par}

- **Comunicação:** troca de dados entre processos/threads.
- **Sincronização:** espera entre tarefas que precisam alinhar resultados.
- **Balanceamento de carga:** divisão desigual de tarefas gera ociosidade em alguns processadores.

Objetivo

- Obter $T_{par} < T_{seq}$. Caso contrário, a paralelização não é vantajosa.

Exemplo didático

Multiplicação de matrizes:

- $T_{seq} = 100s$ (1 processador)
- $T_{par} = 30s$ (4 processadores) → paralelização foi benéfica.

Fundamentos: Speedup, Eficiência e Lei de Amdahl

Objetivo: calcular speedup e eficiência, e usar a Lei de Amdahl para estimar o limite teórico de aceleração.



Tempo de Execução e Speedup

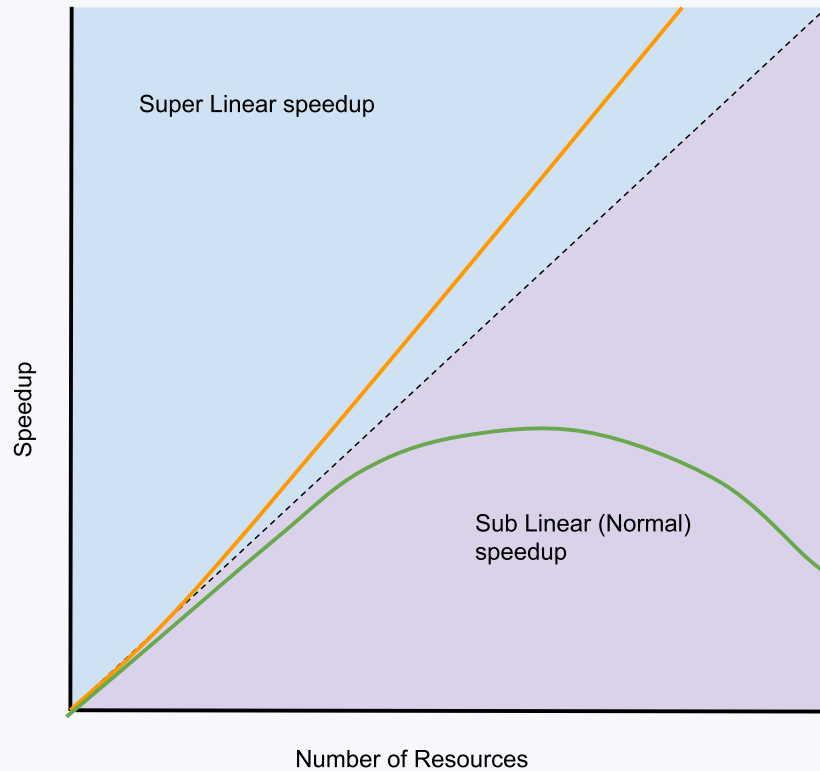
Como medir o ganho de desempenho ao paralelizar um programa

T₁ — tempo do melhor algoritmo sequencial conhecido

T_p — tempo da versão paralela executando em p processadores

$$S(p) = \frac{T_1}{T_p}$$

O speedup mede quantas vezes o programa paralelo é mais rápido que o sequencial.



Linear (ideal)

$S(p) = p$ — cada processador adicionado contribui totalmente



Sublinear

$S(p) < p$ — o mais comum, devido a overhead de comunicação/sincronização



Superlinear

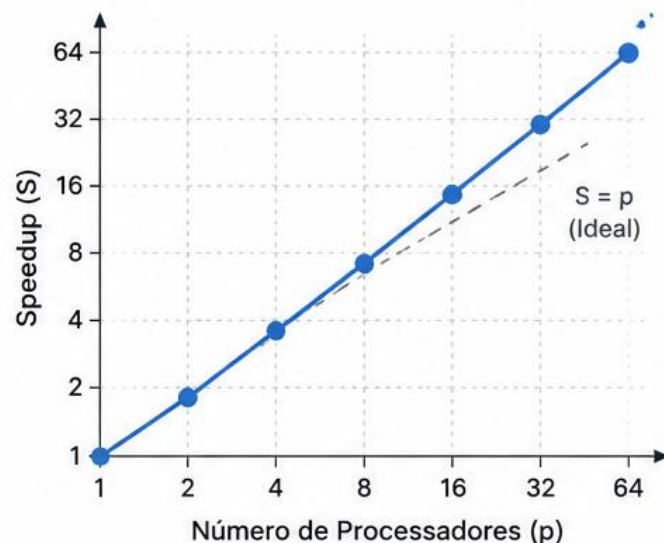
$S(p) > p$ — situação especial, geralmente por efeitos de cache/memória

Comparação de Speeds em Sistemas Paralelos

Speedup (S) = T_1 / T_p onde T_1 é o tempo com 1 processador e T_p é o tempo com p processadores

1) LINEAR (IDEAL)

Speedup cresce proporcionalmente a p



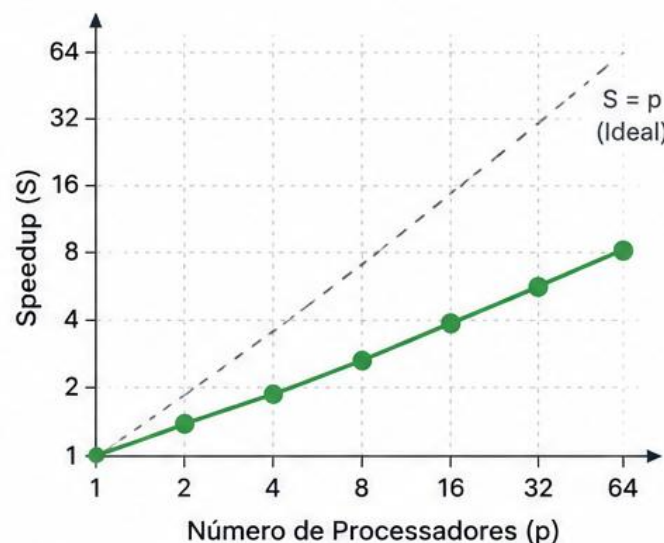
Comportamento Ideal

- Cada dobramento de processadores dobra o speedup.
- Eficiência ($E = S/p$) permanece constante ($= 1$).
- Não há overhead ou interação entre processadores.

Exemplo: problema perfeitamente paralelizável sem overhead.

2) SUBLINEAR

Speedup cresce, mas menos que proporcionalmente



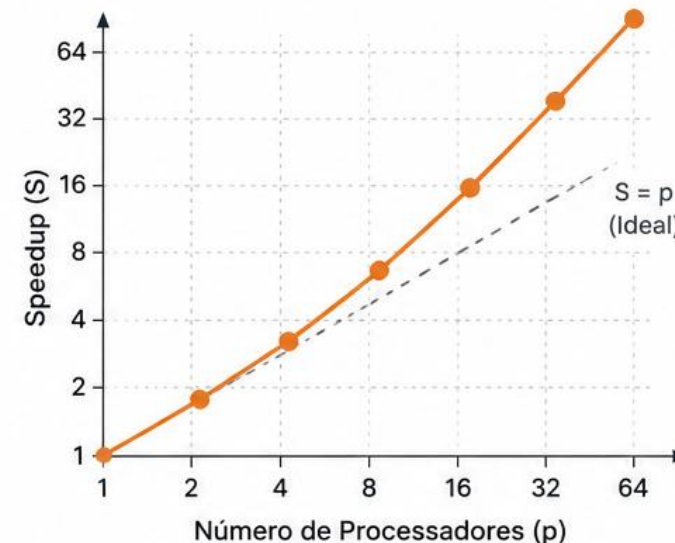
Comportamento Sublinear

- Overhead, comunicação e sincronização limitam o ganho.
- Eficiência diminui à medida que p aumenta.
- Caso mais comum em sistemas reais.

Exemplo: aplicações com comunicação ou parte serial (Lei de Amdahl).

3) SUPERLINEAR

Speedup cresce mais que proporcionalmente



Comportamento Superlinear

- Melhoria adicional além do paralelismo puro.
- Pode ocorrer por melhor uso de cache, redução de I/O, balanceamento de carga, etc.

Exemplo: algoritmos com melhor localidade de dados ou efeitos de cache.



Resumo

$S = p \rightarrow$ linear (ideal) | $S < p \rightarrow$ sublinear (o mais comum) | $S > p \rightarrow$ superlinear (em casos específicos)

Quanto maior o speedup para um dado número de processadores, melhor o desempenho do sistema paralelo.



Fontes de Overhead de Paralelização

Por que o speedup raramente é linear na prática



Comunicação

Troca de mensagens ou dados entre processos/threads, por rede ou barramento de memória.



Sincronização

Tempo ocioso em barreiras e locks, esperando outros processos concluírem uma etapa.



Balanceamento desigual

Processadores rápidos ficam ociosos esperando os mais lentos terminarem.



Contenção de memória/cache

Processadores disputando o mesmo barramento de memória ou linhas de cache ("false sharing").



Gerenciamento de threads/processos

Criação, destruição e escalonamento consomem tempo que não existe na versão sequencial.



Eficiência

O quanto os processadores estão sendo aproveitados

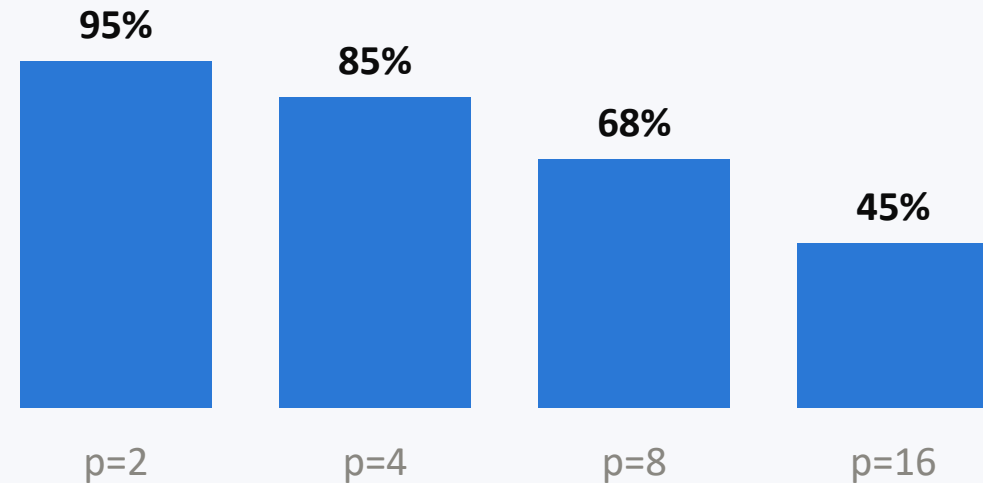
$$E(p) = \frac{S(p)}{p}$$

$E(p)$ representa a fração de tempo em que cada processador realiza trabalho útil, em vez de esperar por comunicação ou sincronização.

$E(p) = 1$ → uso perfeito dos processadores

$E(p) < 1$ → parte da capacidade é perdida com overhead de paralelização

Exemplo visual — eficiência decrescente



Eficiência

Ela relaciona o **speedup obtido** com o **número de processadores** usados:

$$E = \frac{S}{N} = \frac{T_{seq}}{N \cdot T_{par}}$$

Onde:

- E = eficiência
- S = speedup
- N = número de processadores
- T_{seq} = tempo de execução sequencial
- T_{par} = tempo de execução paralelo



Exemplos práticos

- **Caso 1:**

$N = 4$ processadores

$T_{seq} = 100s$

$T_{par} = 30s$

Speedup $S = 100 / 30 \approx 3,33$

Eficiência: $E = 3,33 / 4 \approx 0,833 \rightarrow$
83,3% dos processadores
aproveitados

- **Caso 2:**

$N = 8$ processadores

Speedup $S = 5$

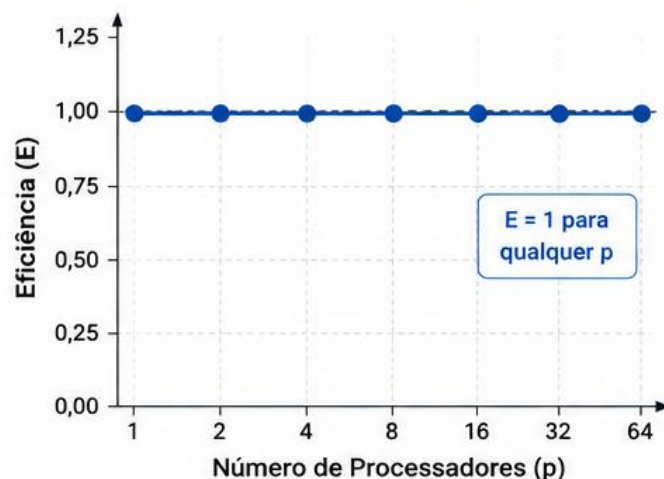
Eficiência: $E = 5 / 8 = 0,625 \rightarrow$
37,5% de recursos
desperdiçados

Comparação de Eficiências em Sistemas Paralelos

Eficiência (E) = $S / p = T_1 / (p \cdot T_p)$ onde S é o speedup, p é o número de processadores

1) LINEAR (IDEAL)

Eficiência permanece constante e igual a 1 (100%)



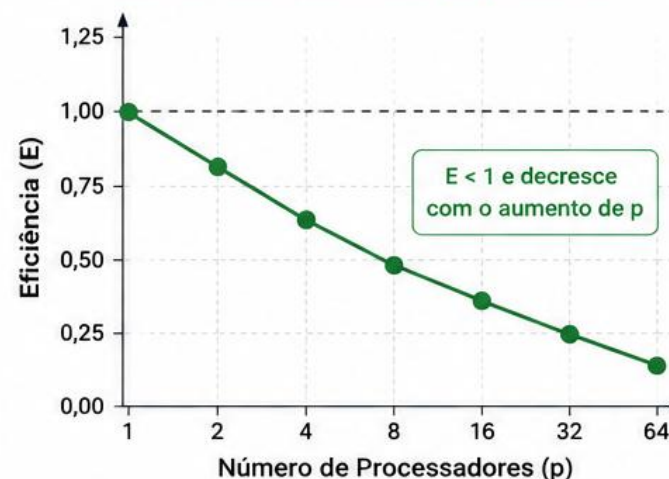
Comportamento Linear (Ideal)

- Speedup cresce proporcionalmente a p ($S = p$).
- Toda a capacidade dos processadores é utilizada.
- Não há overhead ou interação entre processadores.

Exemplo: problema perfeitamente paralelizável sem overhead.

2) SUBLINEAR

Eficiência diminui à medida que p aumenta



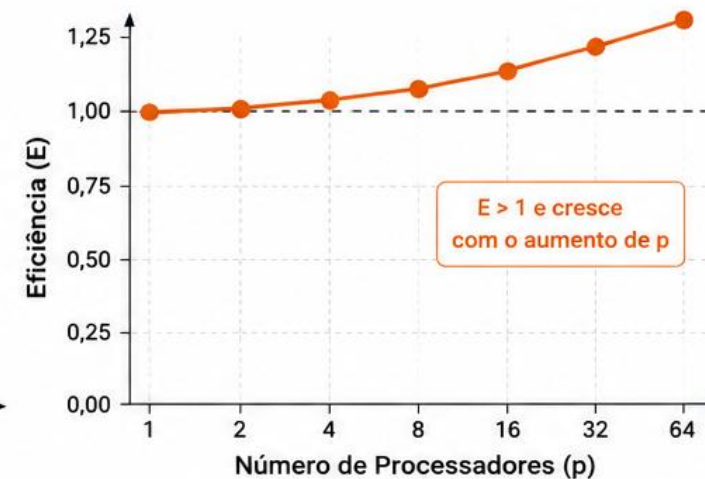
Comportamento Sublinear

- Speedup cresce, mas menos que proporcionalmente ($S < p$).
- Overhead, comunicação e sincronização reduzem a eficiência.
- Caso mais comum em sistemas reais.

Exemplo: aplicações com comunicação, sincronização ou parte serial (Lei de Amdahl).

3) SUPERLINEAR

Eficiência aumenta com o número de processadores ($E > 1$)



Comportamento Superlinear

- Speedup cresce mais que proporcionalmente ($S > p$).
- Ganhos adicionais além do paralelismo puro.
- Pode ocorrer por melhor uso de cache, redução de I/O, balanceamento de carga, etc.

Exemplo: algoritmos com melhor localidade de dados ou efeitos de cache.



Resumo

Linear (Ideal)

E = 1 (100%) constante para qualquer p

Sublinear (mais comum)

0 < E < 1 e decresce à medida que p aumenta

Superlinear (casos específicos)

E > 1 e cresce à medida que p aumenta

Eficiência indica o quanto do potencial teórico está sendo aproveitado na prática.



Lei de Amdahl

O limite teórico do speedup quando o tamanho do problema é fixo

Considera o problema de tamanho fixo e analisa o impacto da fração do programa que não pode ser paralelizada (f).

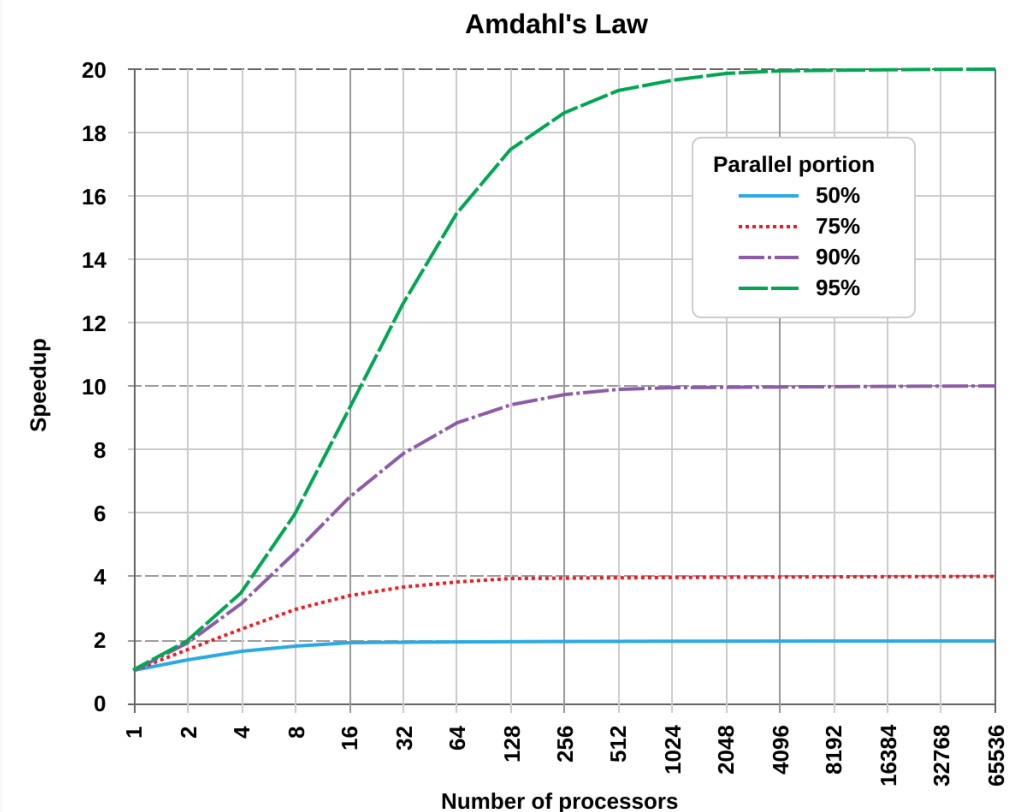
$$S(p) = \frac{1}{f + (1 - f)/p}$$

f = fração estritamente sequencial do programa



Consequência central: mesmo com infinitos processadores, o speedup máximo é limitado a $1/f$. Se 10% do programa é sequencial, o speedup nunca ultrapassa 10.

Speedup se aproxima de um teto ($1/f$)





Dedução da Lei de Amdahl

De onde vem a fórmula — passo a passo

1 Seja f a fração estritamente sequencial e $(1 - f)$ a fração perfeitamente paralelizável do programa.

2 O tempo em p processadores é a soma do tempo da parte sequencial com o tempo da parte paralela dividido por p :

$$T_p = f \cdot T_1 + (1 - f) \cdot \frac{T_1}{p}$$

3 Como $S(p) = T_1 / T_p$, substituindo e colocando T_1 em evidência:

$$S(p) = \frac{1}{f + (1 - f)/p}$$



Quando $p \rightarrow \infty$, o termo $(1 - f)/p$ tende a zero, e o speedup converge para o teto teórico $1/f$.



Exemplo Resolvido — Lei de Amdahl

$f = 0,1$ (10% sequencial). Calcule o speedup para $p = 4, 16$ e 64 .

$p = 4$

$$1 / (0,1 + 0,9/4)$$

$\approx 3,08$

$p = 16$

$$1 / (0,1 + 0,9/16)$$

$\approx 6,40$

$p = 64$

$$1 / (0,1 + 0,9/64)$$

$\approx 8,77$



O ganho marginal diminui rapidamente à medida que p cresce — o speedup se aproxima assintoticamente de 10 ($= 1/f$), mesmo com muito mais processadores.



Exemplo 2 — Sensibilidade à Fração Serial

Reduzir f pela metade quase dobra o teto teórico de speedup

p

$f = 0,10$

$f = 0,05$

$p = 4$

3,08

3,48

$p = 16$

6,40

9,14

$p = 64$

8,77

15,42

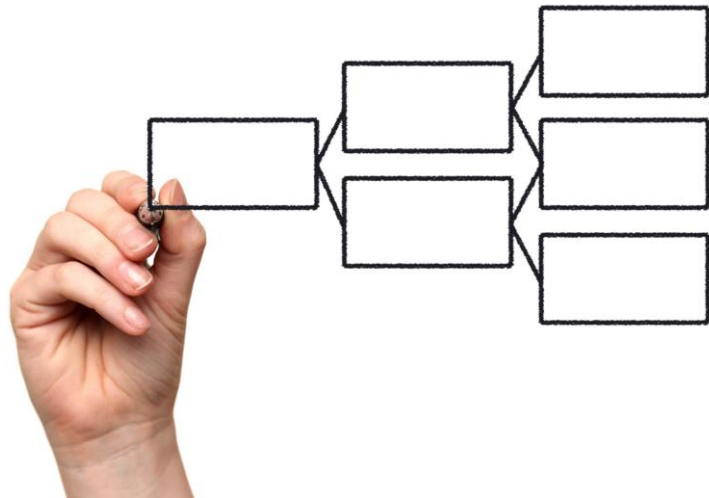


Reduzir f de 0,1 para 0,05 quase dobra o teto teórico (de 10 para 20). Pequenas reduções na fração serial têm impacto desproporcional — um argumento central para otimizar primeiro os trechos sequenciais.

Escalabilidade, Gustafson-Barsis e Métricas Distribuídas

Objetivo: comparar escalabilidade forte e fraca, aplicar a Lei de Gustafson-Barsis e identificar métricas de sistemas distribuídos.

Escalabilidade em Sistemas Paralelos



Escalabilidade

Conceito

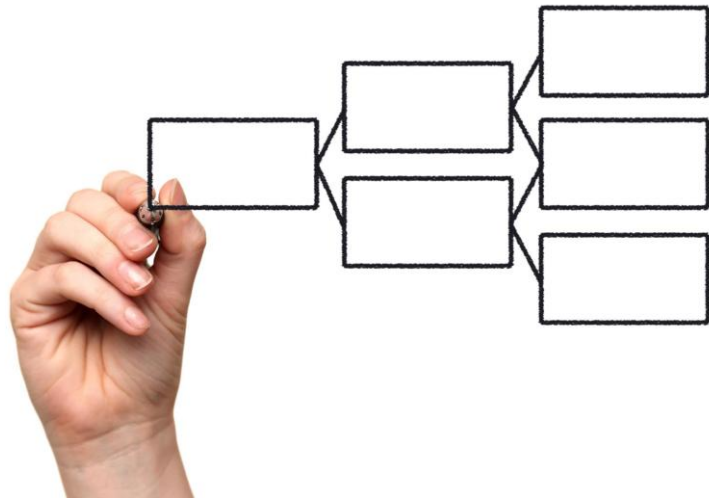
- A **escalabilidade** mede a capacidade de um sistema paralelo **manter bom desempenho** quando aumentamos o número de processadores ou o tamanho do problema.
- Em outras palavras: até que ponto adicionar mais recursos traz **ganhos proporcionais** no desempenho.

Tipos de Escalabilidade

1. Escalabilidade Forte

- **Definição:** o **tamanho do problema é fixo** e aumentamos apenas o número de processadores.
- **Objetivo:** avaliar se o tempo de execução diminui de forma proporcional ao número de processadores.
- **Limitação:** fração sequencial e overhead de comunicação limitam o ganho.
- **Pergunta-chave:** “Posso resolver o mesmo problema mais rápido com mais processadores?”
- **Exemplo:** multiplicação de uma matriz 1000×1000 usando 2, 4, 8 processadores.

Escalabilidade em Sistemas Paralelos



2. Escalabilidade Fraca

- **Definição:** o **tamanho do problema aumenta** proporcionalmente ao número de processadores.
- **Objetivo:** verificar se a eficiência se mantém estável à medida que o problema e os recursos crescem.
- **Pergunta-chave:** “Posso resolver problemas maiores sem perder eficiência à medida que aumento os processadores?”
- **Exemplo:** multiplicar matrizes 1000×1000 com 2 processadores, 2000×2000 com 4 processadores, 4000×4000 com 8 processadores.

Interpretação prática

- Um sistema **fortemente escalável** reduz tempo de execução de um problema fixo quase linearmente com N.
- Um sistema **fraco escalável** mantém a eficiência alta mesmo com problemas maiores e mais processadores.
- Avaliar escalabilidade é importante para **dimensionar clusters, supercomputadores ou aplicações distribuídas**.



Escalabilidade Forte vs. Fraca

Duas formas de analisar o crescimento do desempenho



Escalabilidade Forte

Tamanho do problema fixo; mede-se quão mais rápido ele é resolvido ao adicionar processadores.

Governada pela Lei de Amdahl.

Pergunta-chave: "Quanto mais rápido resolvo o MESMO problema?"



Escalabilidade Fraca

O tamanho do problema cresce proporcionalmente a p , mantendo constante a carga de trabalho por processador.

Governada pela Lei de Gustafson-Barsis.

Pergunta-chave: "Quão MAIOR é o problema que resolvo no mesmo tempo?"



Lei de Gustafson-Barsis

Speedup escalado: o problema cresce junto com os processadores

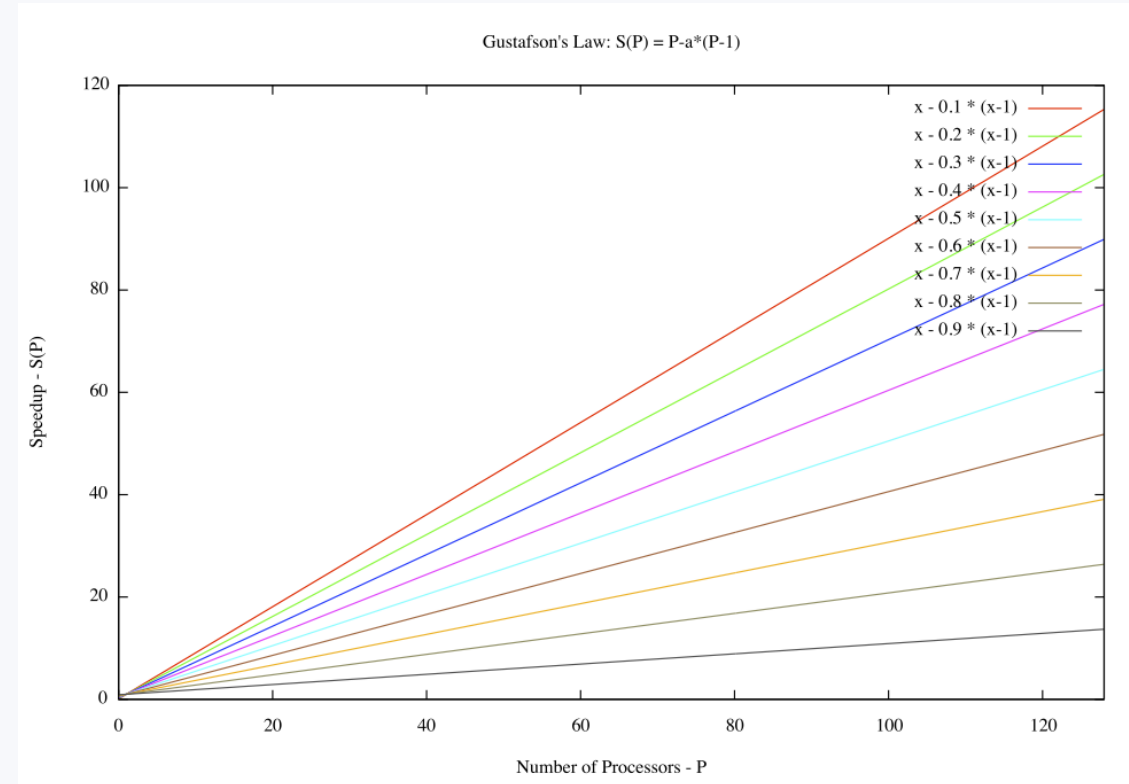
Parte da premissa de que, na prática, mais poder computacional é usado para resolver problemas maiores — não apenas o mesmo problema mais rápido.

$$S(p) = p - f \times (p - 1)$$

f = fração serial medida na execução paralela



Diferença chave: o speedup escalado cresce de forma aproximadamente linear com p, pois a fração serial passa a representar uma parcela cada vez menor do trabalho total à medida que o problema cresce.





Dedução da Lei de Gustafson-Barsis

De onde vem o speedup escalado

Seja f a fração serial medida na execução paralela (do problema já escalado). Normalizando o tempo paralelo total para 1 unidade:

- 1 A parte serial (f) não se beneficia do paralelismo — seu tempo sequencial equivalente também é f .
- 2 A parte paralela ($1 - f$) levaria p vezes mais tempo se executada sequencialmente em 1 processador: $(1 - f) \cdot p$.

O speedup escalado é a razão entre o tempo sequencial equivalente e o tempo paralelo (normalizado a 1):

$$S(p) = f + (1 - f) \cdot p = p - f \cdot (p - 1)$$



Diferente de Amdahl, aqui não há termo dividido por p limitando o crescimento — o speedup escalado cresce quase linearmente com p .



Nota: Modelo de Sun-Ni (Memory-Bounded Speedup)

Uma extensão que unifica Amdahl e Gustafson — leitura complementar



O modelo de Sun-Ni assume que o tamanho do problema cresce até o limite da memória disponível em cada configuração — nem fixo (Amdahl), nem crescendo livremente (Gustafson), mas limitado pela capacidade de memória de p processadores.

Amdahl e Gustafson-Barsis são, portanto, dois casos particulares de um espectro mais amplo de modelos de escalabilidade — não é necessário aprofundar a fórmula em aula.



Comparação: Amdahl x Gustafson-Barsis

Mesma fração serial ($f = 0,1$), $p = 64$ — conclusões muito diferentes

AMDAHL

(problema fixo)

8,77

speedup em $p = 64$

Apenas 14% do ganho ideal de 64x

GUSTAFSON-BARSIS

(problema escalado)

57,7

speedup escalado em $p = 64$

90% do ganho ideal de 64x



Métrica de Karp-Flatt

Estimando a fração serial a partir de medições reais

Permite calcular a fração serial experimental (e) a partir de medições reais de speedup — sem assumir seu valor a priori.

$$e = \frac{1/S(p) - 1/p}{1 - 1/p}$$



Por que é útil: diagnostica se a perda de desempenho vem de overhead de paralelização (comunicação, sincronização) — que cresce com p — e não apenas da fração serial fixa do programa. Algo que nem Amdahl nem Gustafson capturam diretamente.



Exemplo Resolvido — Karp-Flatt

Duas medições reais: $p = 4$ ($S = 3,2$) e $p = 8$ ($S = 5,33$)

$$p = 4, S = 3,2$$

$$(1/3,2 - 1/4) / (1 - 1/4)$$

$$e \approx 0,083$$

$$p = 8, S = 5,33$$

$$(1/5,33 - 1/8) / (1 - 1/8)$$

$$e \approx 0,072$$



e permaneceu praticamente estável entre $p = 4$ e $p = 8$ ($0,083 \rightarrow 0,072$) — bom sinal: o overhead de paralelização não está crescendo significativamente. Se e aumentasse nitidamente com p , isso apontaria overhead de comunicação/sincronização crescente, um alerta de má escalabilidade.

Studying Scalability

Table records the parallel runtime (in seconds) for varying values of n and p.

Input size (n)	Number of threads (p)				
	1	2	4	8	16
1,000					
2,000					
4,000					
8,000					
16,000					

It is conventional to test scalability in powers of two (or by doubling n and p).

Studying Scalability

Table records the parallel runtime (in seconds) for varying values of n and p .

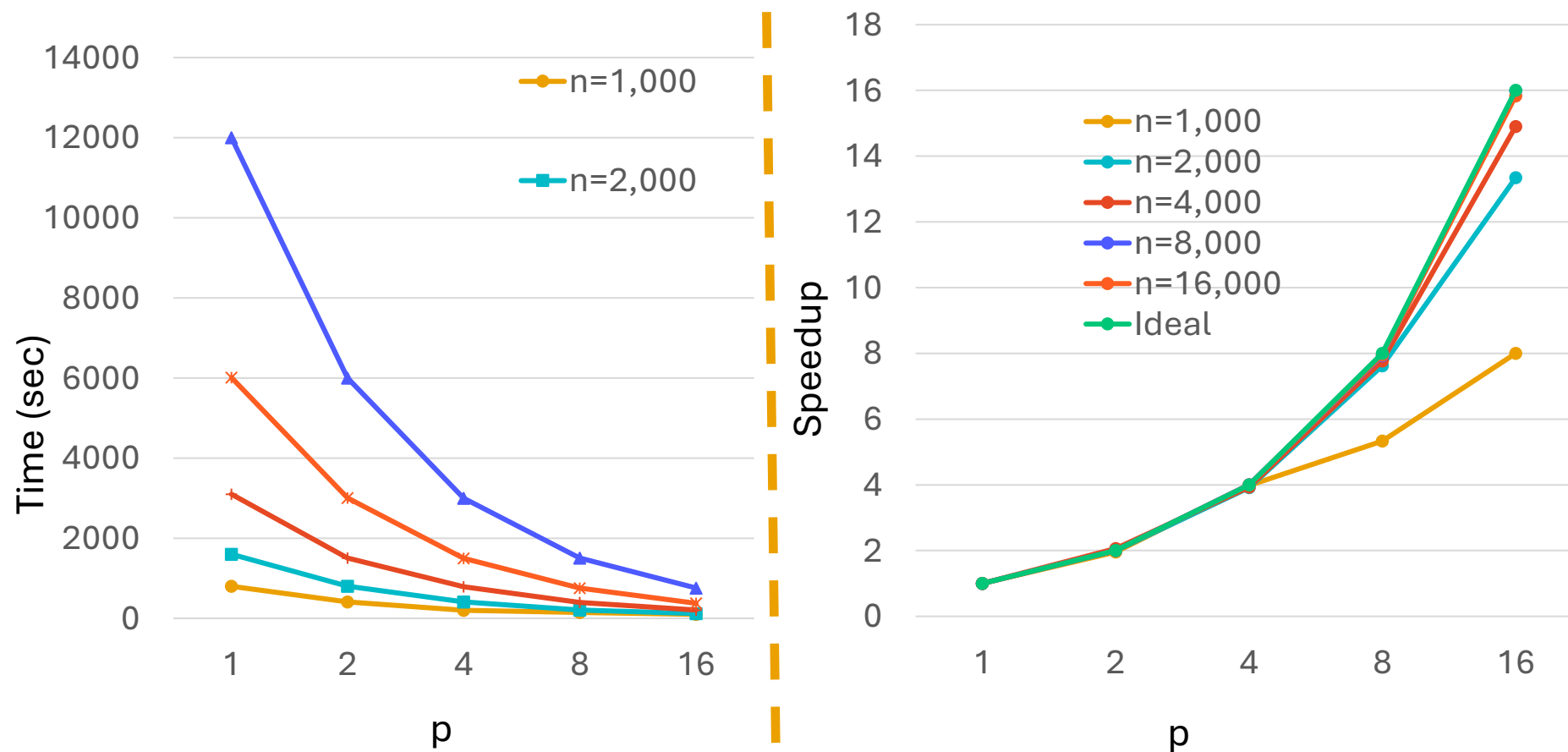
Input size (n)	Number of threads (p)				
	1	2	4	8	16
1,000	800	410	201	150	100
2,000	1,601	802	409	210	120
4,000	3,100	1,504	789	399	208
8,000	6,010	3,005	1,500	758	376
16,000	12,000	6,000	3,001	1,509	758

Strong scaling
behavior

Weak scaling
behavior

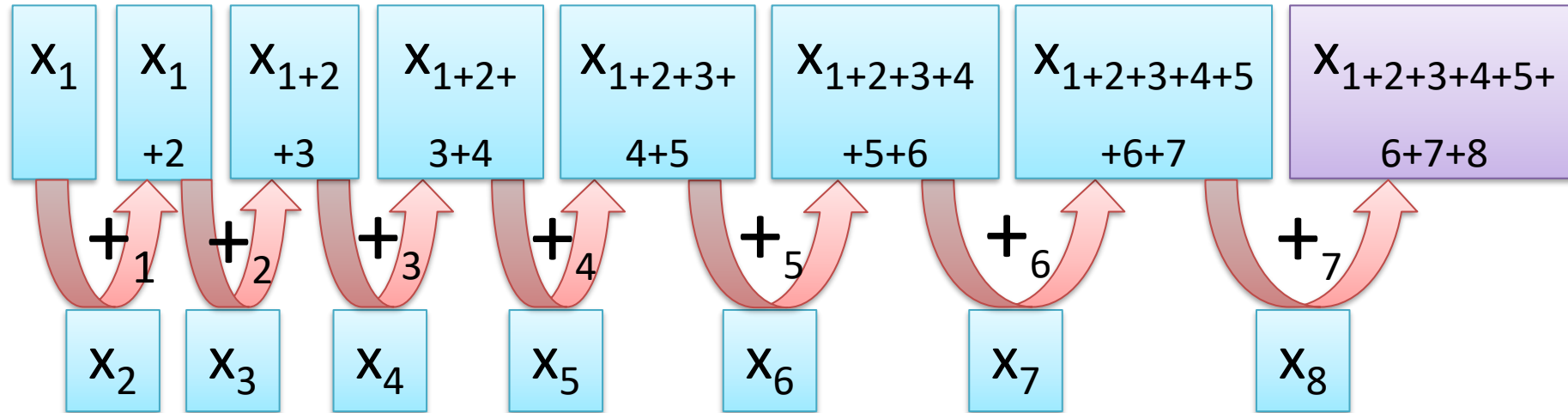
It is conventional to test scalability in powers of two (or by doubling n and p).

Studying Scalability

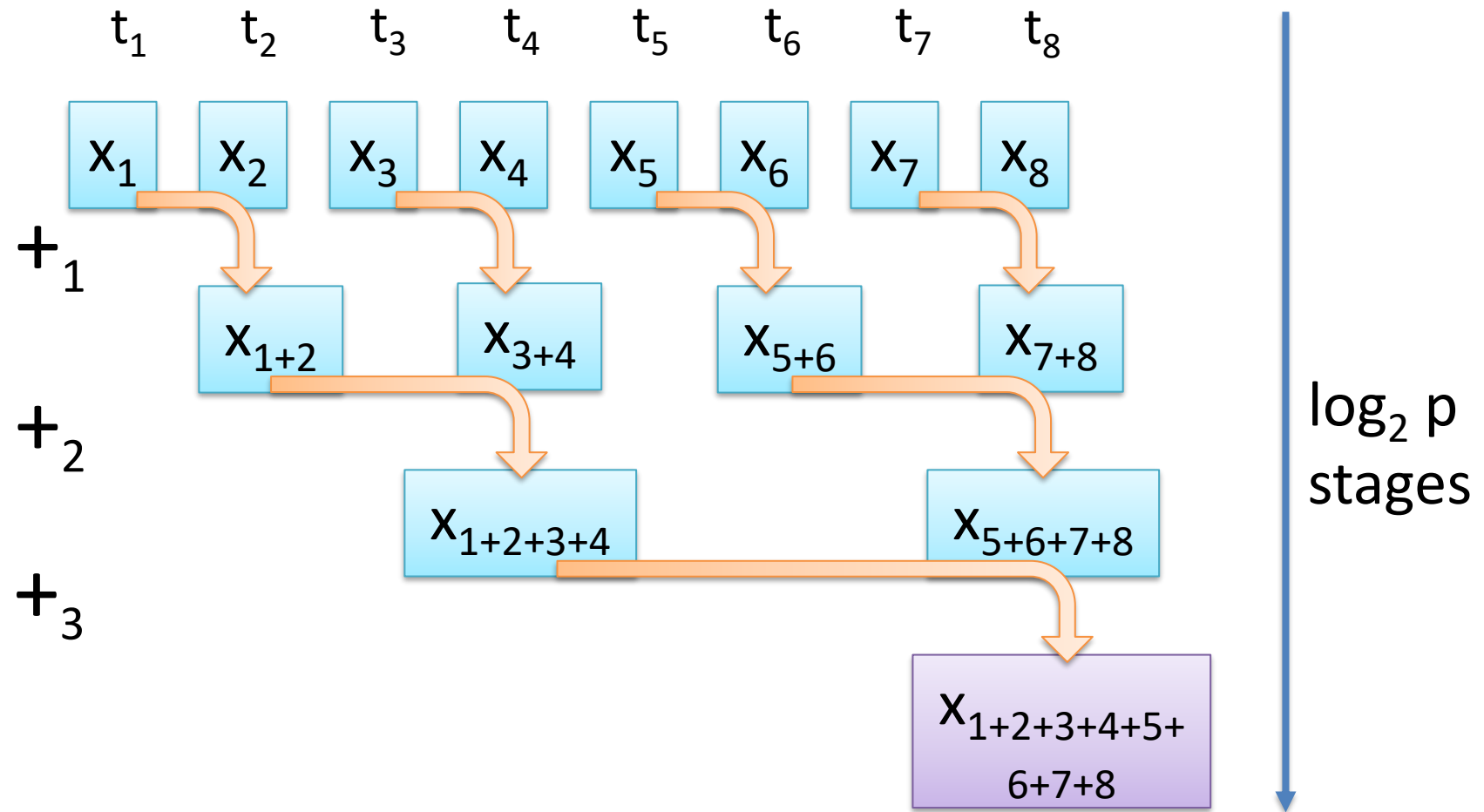


Serial vs. Parallel Reduction

Serial Process (1 thread, 7 operations)



Parallel Process (8 threads, 3 operations)



Exercício

Analisando paralelismo em múltiplos cenários

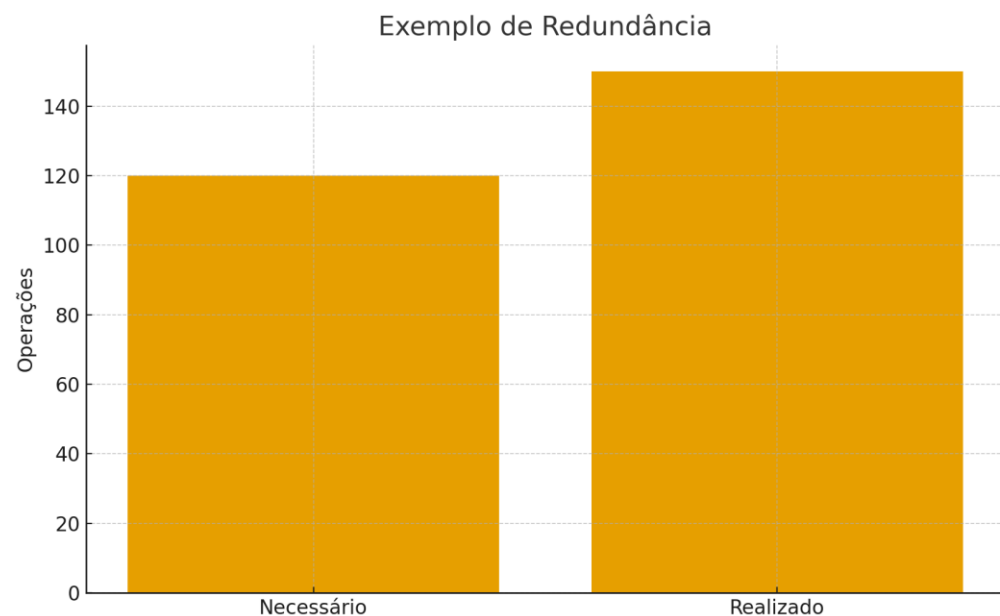
- Um programa sequencial leva **300 segundos** para ser executado. Ele é testado em diferentes números de processadores e os tempos obtidos foram:

Processadores (N)	Tempo de execução (Tpar)
1	300 s
2	160 s
4	90 s
8	55 s

Perguntas:

- Calcule o **speedup** para $N = 2, 4$ e 8 processadores.
- Calcule a **eficiência** para $N = 2, 4$ e 8 processadores.
- Avalie se o programa apresenta **boa escalabilidade forte**. Justifique.
- Suponha que você possa dobrar o tamanho do problema para cada aumento de processador (escala fraca). Discuta qualitativamente se a eficiência poderia aumentar, diminuir ou se manter.

Redundância



Mede o trabalho extra realizado além do necessário.

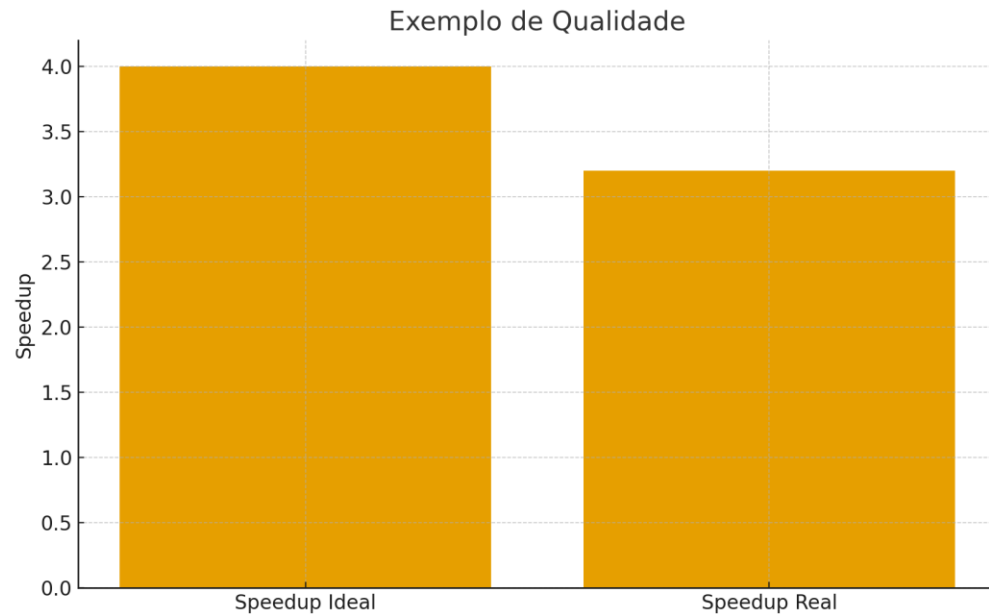
Ligada ao overhead do paralelismo (duplicações, comunicação).

Fórmula:

$$R = T_{\text{total_paralelo}} / T_{\text{necessário}}$$

Exemplo: 1500 operações realizadas / 1200 necessárias = $R = 1,25$

Qualidade (Quality)



Mede o ganho real do paralelismo considerando redundância.

Avalia a proximidade do desempenho real ao ideal.

Fórmula:

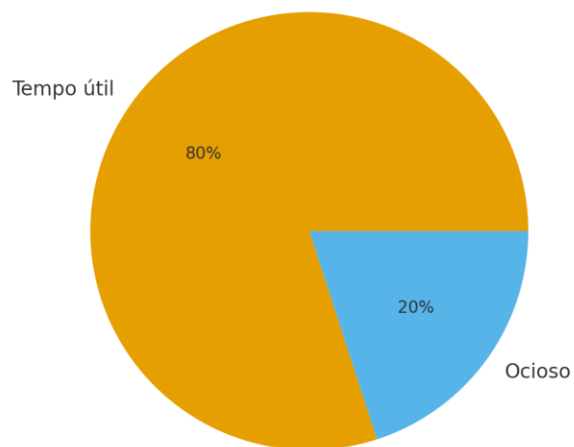
$$Q = \text{Speedup} / \text{Redundância}$$

Interpretação:

- $Q \approx 1 \rightarrow$ alto aproveitamento do paralelismo.
- $Q < 1 \rightarrow$ perdas por overhead ou redundância.

Utilização

Exemplo de Utilização dos Processadores



Mede quanto do tempo total dos processadores é realmente usado.

Avalia ociosidade por espera, comunicação ou sincronização.

Fórmula:

$$U = T_{\text{compute}} / (N \times T_{\text{par}})$$

Interpretação:

- $U \approx 1 \rightarrow$ uso ótimo dos processadores.
- U baixo $\rightarrow$ muitos recursos ociosos.

Conclusão

Desempenho vai além do tempo de execução

Throughput, tempo de resposta, speedup e eficiência oferecem visões complementares sobre o comportamento de um sistema paralelo.

Overhead explica o speedup real

Comunicação, sincronização, contenção e balanceamento de carga são os principais fatores que afastam o desempenho do ganho ideal.

Amdahl x Gustafson-Barsis

Escalabilidade forte (problema fixo) e fraca (problema crescente) revelam limites teóricos muito diferentes para o ganho de paralelização.

Medir para otimizar

Métricas como Karp-Flatt, redundância, qualidade e utilização permitem diagnosticar perdas reais de desempenho e orientar decisões de projeto.

